# OTEL-06: Logs and Events

> **Series:** OTEL — OpenTelemetry Integration | **Notebook:** 6 of 8 | **Created:** January 2026 | **Last Updated:** 07/01/2026

## Integrating Logs with OpenTelemetry

OpenTelemetry logs bridge traditional logging with distributed tracing, enabling correlation between log events and traces. This notebook covers log instrumentation, correlation, and best practices.

---

## Table of Contents



---

## Prerequisites

| Requirement | Details |
| --- | --- |
| **Knowledge** | OTEL-01 and OTEL-04 |
| **Environment** | Collector with logs pipeline |

# 1. OTel Logs Overview

## Log Data Model

| Field | Description | Example |
|---|---|---|
| `timestamp` | When event occurred | `2026-01-26T10:00:00Z` |
| `observed_timestamp` | When log was collected | `2026-01-26T10:00:01Z` |
| `severity_number` | Numeric severity (1-24) | `17` (ERROR) |
| `severity_text` | Severity string | `ERROR` |
| `body` | Log message | `Failed to connect` |
| `attributes` | Structured fields | `{"user.id": "123"}` |
| `resource` | Resource attributes | `{"service.name": "api"}` |
| `trace_id` | Associated trace | `abc123...` |
| `span_id` | Associated span | `def456...` |

## Severity Levels

| Number | Level | Use Case |
|---|---|---|
| 1-4 | TRACE | Detailed debugging |
| 5-8 | DEBUG | Development info |
| 9-12 | INFO | Normal operations |
| 13-16 | WARN | Potential issues |
| 17-20 | ERROR | Errors occurred |
| 21-24 | FATAL | Critical failures |

# 2. Log Bridge Pattern

OTel doesn't replace your logging library—it bridges logs to OTel format.

## Python Logging Bridge

```python
import logging
from opentelemetry import _logs
from opentelemetry.sdk._logs import LoggerProvider, LoggingHandler
from opentelemetry.sdk._logs.export import BatchLogRecordProcessor
from opentelemetry.exporter.otlp.proto.grpc._log_exporter import
OTLPLogExporter

# Setup OTel logs
logger_provider = LoggerProvider()
exporter = OTLPLogExporter(endpoint="http://collector:4317")
logger_provider.add_log_record_processor(BatchLogRecordProcessor(exporter))
_logs.set_logger_provider(logger_provider)

# Bridge to Python logging
handler = LoggingHandler(
    level=logging.INFO,
    logger_provider=logger_provider
)

# Attach to standard logger
logging.getLogger().addHandler(handler)
logging.getLogger().setLevel(logging.INFO)

# Use standard logging - logs go to OTel
logger = logging.getLogger(__name__)
logger.info("Application started")
logger.error("Connection failed", extra={"host": "db.example.com"})
```

## Java Log4j Bridge

```
&lt;!-- pom.xml --&gt;
&lt;dependency&gt;
    &lt;groupId&gt;io.opentelemetry.instrumentation&lt;/groupId&gt;
    &lt;artifactId&gt;opentelemetry-log4j-appender-2.17&lt;/artifactId&gt;
    &lt;version&gt;1.26.0-alpha&lt;/version&gt;
&lt;/dependency&gt;
```

```
&lt;!-- log4j2.xml --&gt;
&lt;Appenders&gt;
    &lt;OpenTelemetry name="OpenTelemetry" /&gt;
&lt;/Appenders&gt;
&lt;Loggers&gt;
    &lt;Root level="info"&gt;
        &lt;AppenderRef ref="OpenTelemetry" /&gt;
    &lt;/Root&gt;
&lt;/Loggers&gt;
```

# 3. Log-Trace Correlation

## Automatic Correlation

When using OTel tracing with the log bridge, trace context is automatically added:

```
from opentelemetry import trace
import logging

logger = logging.getLogger(__name__)
tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span("process_order"):
    # Log automatically gets trace_id and span_id
    logger.info("Processing order", extra={"order_id": "12345"})
```

## Manual Correlation

If not using OTel log bridge:

```
from opentelemetry import trace

def get_trace_context():
    span = trace.get_current_span()
    ctx = span.get_span_context()
    return {
        "trace_id": format(ctx.trace_id, '032x'),
        "span_id": format(ctx.span_id, '016x')
    }

# Add to log format
ctx = get_trace_context()
logger.info(f"[trace_id={ctx['trace_id']}] Processing order")
```

## Correlation in Dynatrace

When logs include trace_id:
- View logs from trace details
- Jump to trace from log entry
- Filter logs by trace context

```
// Find logs with trace correlation
fetch logs, from:-1h
| filter isNotNull(trace_id)
| fields timestamp, trace_id, span_id, content, loglevel
| sort timestamp desc
| limit 20
```

```
// Find logs for a specific trace
fetch logs, from:-1h
| filter trace_id == "REPLACE_WITH_TRACE_ID"
| fields timestamp, span_id, content, loglevel
| sort timestamp asc
```

# 4. Collector Log Processing

## OTLP Log Receiver

```yaml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318
```

## Log Processing

```yaml
processors:
  # Add resource attributes
  resource:
    attributes:
      - key: environment
        value: production
        action: upsert

  # Parse JSON logs
  transform:
    log_statements:
      - context: log
        statements:
          - merge_maps(attributes, ParseJSON(body), "insert")

  # Filter debug logs
  filter:
    logs:
      log_record:
        - 'severity_number &lt; 9'  # Drop TRACE and DEBUG

service:
  pipelines:
    logs:
      receivers: [otlp]
      processors: [resource, transform, filter, batch]
      exporters: [otlphttp]
```

# 5. File-Based Log Collection

### File Log Receiver (`file_log`, formerly `filelog` — old name kept as a deprecated alias)

```yaml
receivers:
  file_log:
    include:
      - /var/log/app/*.log
    exclude:
      - /var/log/app/debug.log
    start_at: beginning
    include_file_path: true
    include_file_name: true
    operators:
      # Parse JSON logs
      - type: json_parser
        if: 'body matches "^\\{"'

      # Parse timestamp
      - type: time_parser
        parse_from: attributes.timestamp
        layout: '%Y-%m-%dT%H:%M:%S.%LZ'

      # Set severity
      - type: severity_parser
        parse_from: attributes.level
        mapping:
          error: error
          warn: warn
          info: info
          debug: debug
```

## Container Logs

```
receivers:
  file_log:
    include:
      - /var/log/containers/*.log
    operators:
      # Parse container log format
      - type: regex_parser
        regex: '^(?P&lt;time&gt;[^ ]+) (?P&lt;stream&gt;stdout|stderr) (?P&lt;logtag&gt;[^ ]*) ?(?P&lt;log&gt;.*)$'

      # Extract k8s metadata from filename
      - type: regex_parser
        parse_from: attributes["log.file.name"]
        regex: '^(?P&lt;pod_name&gt;[^_]+)_(?P&lt;namespace&gt;[^_]+)_(?P&lt;container&gt;.+)\.log$'
```

# 6. Structured Logging

## JSON Logging

```
import json
import logging

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_obj = {
            "timestamp": self.formatTime(record),
            "level": record.levelname,
            "message": record.getMessage(),
            "logger": record.name,
        }
        if hasattr(record, 'extra_fields'):
            log_obj.update(record.extra_fields)
        return json.dumps(log_obj)

# Use JSON formatter
handler = logging.StreamHandler()
handler.setFormatter(JSONFormatter())
logger.addHandler(handler)

# Log with extra fields
logger.info("Order processed", extra={"extra_fields": {
    "order_id": "12345",
    "customer_id": "c-789",
    "total": 99.99
}})
```

### structlog

```
import structlog

structlog.configure(
    processors=[
        structlog.processors.add_log_level,
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.JSONRenderer()
    ]
)

logger = structlog.get_logger()
logger.info("order_processed", order_id="12345", total=99.99)
```

## 7. Best Practices

### Log Content

| Do | Don't |
|---|---|
| Use structured fields | Log PII or secrets |
| Include relevant context | Log at DEBUG in prod |
| Add trace correlation | Use inconsistent levels |
| Log actionable events | Log every detail |

### Performance

| Tip | Why |
|---|---|
| Batch log exports | Reduce network calls |
| Filter at source | Reduce volume |
| Use async logging | Don't block requests |
| Set appropriate levels | Control verbosity |

### Log Levels Guide

| Level | When to Use | Example |
|---|---|---|
| ERROR | Unexpected failures | DB connection failed |
| WARN | Recoverable issues | Retry succeeded |
| INFO | Business events | Order completed |
| DEBUG | Development details | Query parameters |

---

## Summary

In this notebook, you learned:

- OTel logs data model and severity levels
- Log bridge pattern for existing loggers
- Automatic and manual log-trace correlation
- Collector log processing and transformation
- File-based log collection
- Structured logging best practices

---

## References

- OTel Logs Specification
- Filelog Receiver
- Dynatrace OTel Log Ingest

---